Viet Nam National University, HCMC

University of Science

Institution of Information Technology

Advanced Program in Computer Science

Project 2: Machine Learning

Subject: Artificial Intelligence

Student:

Ho Lam Ngoc Bao

Huynh Tran Uy

Nguyen Hoang Gia Han

Tran My An

Lecturer:

Dr. Nguyen Ngoc Thao

MSc. Nguyen Thanh Tinh

March 3, 2026

Contents

1 Project Planning and Task Distribution **2**

2 Introduction **2**

3 MNIST Dataset **3**

3.1 Preparing the Dataset 3

3.2 Building the Decision Tree Classifier 4

3.3 Hyperparameter Tuning for Decision Tree 4

3.4 Neural Network Classifier (MLP) 5

3.5 Test Evaluation, Interpretation, and Comparison 6

4 Fashion-MNIST Dataset **9**

4.1 Preparing the Dataset 9

4.2 Building the Decision Tree Classifier 10

4.3 Hyperparameter Tuning for Decision Tree 10

4.4 Neural Network Classifier (MLP) 11

4.5 Test Evaluation and Interpretation 12

5 Food-MNIST Dataset **14**

5.1 Preparing the Dataset 14

5.2 Building the Decision Tree Classifier 15

5.3 Hyperparameter Tuning for Decision Tree 15

5.4 Neural Network Classifier (MLP) 16

5.5 Test Evaluation and Interpretation 17

5.6 Bonus Task: Convolution Neural Network (CNN) 19

6 Model Comparison and Performance Analysis **20**

7 Conclusions **21**

References **22**

1 Project Planning and Task Distribution

Member	Student ID	Contribution (%)	Note
Huynh Tran Uy	23125047	100%	MNIST, Fashion-MNIST, Report
Tran My An	23125049	100%	MNIST, Food-MNIST, Report
Ho Lam Ngoc Bao	23125050	100%	Fashion-MNIST, Food-MNIST, Report
Nguyen Hoang Gia Han	23125055	100%	Fashion-MNIST, Food-MNIST, Report

Table 1: Contribution percentage of team members.

2 Introduction

This project [2] investigates supervised learning using `scikit-learn` on three image classification datasets: **MNIST** [1] , **Fashion-MNIST** [4], and **Food-MNIST** [3]. Two supervised models are implemented and compared:

- **Decision Tree Classifier** (criterion = entropy / information gain).
- **Neural Network Classifier** using **MLPClassifier** (multi-layer perceptron).

For each dataset, the workflow strictly follows the project specifications:

1. Prepare data: stratified shuffle split (80% train, 20% validation) while keeping the test set untouched.
2. Train a decision tree and visualize it using Graphviz.
3. Tune decision tree hyperparameters and select the best configuration using validation accuracy.
4. Train a neural network (MLPClassifier) and evaluate validation accuracy.
5. Evaluate on the test set: classification report and confusion matrix; interpret results and compare models.

3 MNIST Dataset

3.1 Preparing the Dataset

Data representation. Each MNIST image is a 28×28 grayscale image. In the notebook, each image is flattened into a vector in $\mathbb{R}^{784}$ (since $28 \times 28 = 784$). Thus, the classification problem becomes a standard multi-class supervised learning task with 10 classes (digits 0–9), where each sample is a 784-dimensional feature vector.

Train/Validation split (Stratified 80/20). Following the requirement, the original training set is first shuffled and then split into:

- `feature_train`, `label_train`: 80% for model fitting.
- `feature_val`, `label_val`: 20% for hyperparameter selection.

A *stratified* strategy is used, meaning the class proportion of digits in each subset is preserved. This is important because validation accuracy would be misleading if some digits were underrepresented in the validation set.

Split results from the notebook.

Subset	Shape (features)	Notes
Training	$(48000, 784)$	stratified 80% of original training
Validation	$(12000, 784)$	stratified 20% of original training
Test	$(10000, 784)$	untouched, used only for final evaluation

Class distribution check. The notebook visualizes class distributions for the original training data, training split, validation split, and test set. The bar charts confirm that the split is properly stratified and does not introduce class imbalance.

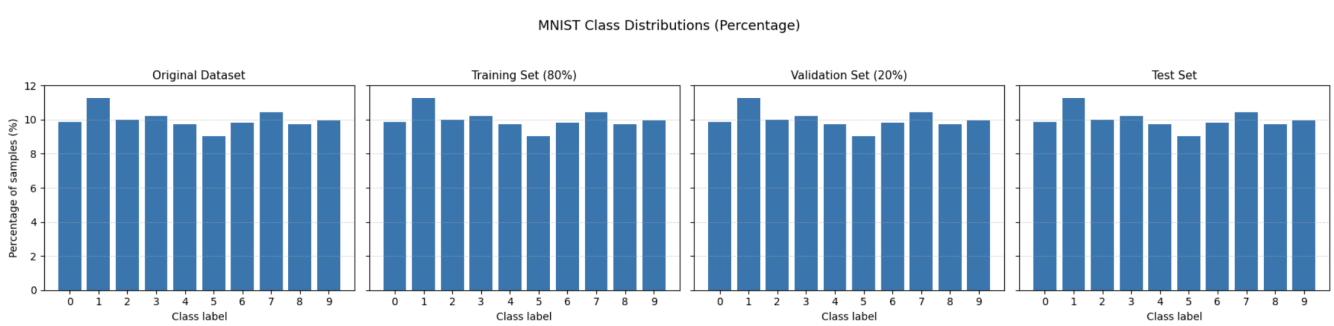

Figure 1: MNIST class distribution across original training, train (80%), validation (20%), and test. (Full-resolution visualization is available in the MNIST notebook.)

3.2 Building the Decision Tree Classifier

Model choice. We trained a decision tree using information gain by setting:

```
DecisionTreeClassifier(criterion='entropy')
```

Using entropy means the tree selects splits that maximize information gain, i.e., it prefers splits that most reduce class impurity.

Why decision trees may struggle on pixel features. Although decision trees are interpretable, using raw pixels as independent features makes the problem challenging:

- Each pixel is treated as one feature, leading to a high-dimensional input space (784 features).
- Tree splits are *axis-aligned* (one feature threshold at a time), which is not ideal for capturing global shapes (strokes and curves) that depend on combinations of many pixels.
- To fit complex image patterns, trees often grow large and can overfit if not constrained.

3.3 Hyperparameter Tuning for Decision Tree

Tuning methodology (manual grid search). In the notebook, we performed a manual grid search using nested loops. For each hyperparameter combination:

$$(\texttt{max_depth},\ \texttt{min_samples_split},\ \texttt{min_samples_leaf}),$$

we trained on the training set and computed validation accuracy on the validation set. The best configuration is chosen by:

$$\theta^* = \arg\max_{\theta}\ \text{Accuracy}_{val}(\theta).$$

Candidate sets (as implemented).

$$\texttt{max_depth} \in \{10, 20, 30\}, \quad \texttt{min_samples_split} \in \{2, 10, 50\}, \quad \texttt{min_samples_leaf} \in \{1, 5, 10\}.$$

Why these hyperparameters matter.

- `max_depth`: controls how many sequential splits are allowed. Deeper trees can fit more complex decision boundaries but are more likely to overfit.
- `min_samples_split`: prevents splitting nodes that contain too few samples. Larger values make the tree more conservative, improving generalization.
- `min_samples_leaf`: enforces a minimum leaf size. Increasing this value reduces variance by avoiding tiny leaves that memorize noise.

Best validation result (from notebook output).

Metric	Value	Interpretation
Best validation accuracy	**0.8790**	best DT performance on validation set
Best parameters	**(20, 2, 1)**	(max_depth, min_split, min_leaf)

Interpretation of the selected parameters.

- `max_depth=20`: provides enough capacity to capture digit structure (curves, strokes) better than shallow trees.
- `min_samples_split=2` and `min_samples_leaf=1`: allow fine-grained splitting, which can improve accuracy on MNIST because the dataset is relatively clean and well-aligned.

However, even with tuning, decision trees are still limited for image data compared to neural networks.

3.4 Neural Network Classifier (MLP)

Architecture and hyperparameters We trained an MLP classifier with:

- Hidden layers: `(128,)` (one hidden layer with 128 neurons)
- Activation: `relu`
- Optimizer (solver): `adam`
- `max_iter = 100`
- **Learning rate strategy:** constant (default), with `learning_rate_init=0.001` (default)

- `random_state = 42` (for reproducibility)

Why this architecture makes sense for MNIST.

- A hidden layer enables learning nonlinear combinations of pixels, which is essential for recognizing shapes.
- 128 neurons provide a reasonable trade-off between model capacity and training time.
- ReLU is computationally efficient and helps training deep nonlinear models.
- Adam is a robust optimizer that adapts learning rates automatically.

Validation result

$$\textbf{Validation Accuracy (MLP)} = 0.9790833$$

This is a substantial improvement over the tuned decision tree (0.8790), indicating the MLP learns more discriminative representations from pixel inputs.

3.5 Test Evaluation, Interpretation, and Comparison

Evaluation protocol. After selecting the best decision tree hyperparameters and training the MLP, we evaluated both models on the untouched test set:

- `classification_report`: precision, recall, and F1-score per digit.
- `confusion_matrix`: detailed breakdown of which digits are confused with others.

Classification Report:

Decision Tree — Classification Report

	precision	recall	f1-score	support
0	0.9145	0.9219	0.9182	986
1	0.9473	0.9742	0.9606	1125
2	0.8796	0.8408	0.8598	999
3	0.8428	0.8461	0.8444	1020
4	0.8629	0.8841	0.8734	975
5	0.8263	0.8282	0.8272	902
6	0.9106	0.9022	0.9064	982
7	0.9200	0.9165	0.9183	1042
8	0.8378	0.8318	0.8348	975
9	0.8441	0.8390	0.8416	994
accuracy			0.8803	10000
macro avg	0.8786	0.8785	0.8785	10000
weighted avg	0.8801	0.8803	0.8801	10000

Neural Network (MLP) — Classification Report

	precision	recall	f1-score	support
0	0.9799	0.9909	0.9854	986
1	0.9867	0.9902	0.9885	1125
2	0.9760	0.9790	0.9775	999
3	0.9782	0.9657	0.9719	1020
4	0.9795	0.9795	0.9795	975
5	0.9745	0.9734	0.9739	902
6	0.9878	0.9898	0.9888	982
7	0.9768	0.9712	0.9740	1042
8	0.9692	0.9682	0.9687	975
9	0.9648	0.9658	0.9653	994
accuracy			0.9775	10000
macro avg	0.9773	0.9774	0.9773	10000
weighted avg	0.9775	0.9775	0.9775	10000

Figure 2: MNIST - Classification Reports

Observations. The MLP achieves a substantially higher overall performance compared to the decision tree. The decision tree reaches an overall accuracy of about **88.03%**, while the MLP achieves approximately **97.75%**. Across most digits, the MLP maintains consistently high precision and recall (generally above 0.96), indicating strong generalization. In contrast, the decision tree shows noticeably weaker recall for several classes, meaning it tends to miss true samples more frequently. This difference highlights the MLP's ability to learn more discriminative and robust representations from pixel-level inputs.

Confusion Matrices:

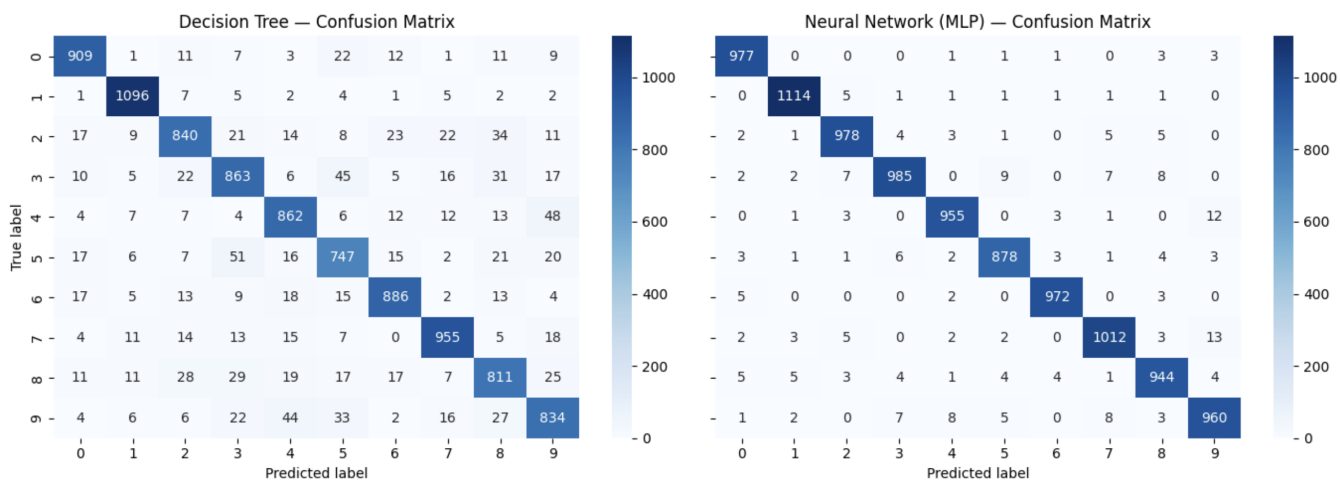

Figure 3: MNIST - Confusion Matrices

Observations Both confusion matrices show strong diagonal dominance, meaning most predictions are correct. However, the decision tree produces more off-diagonal errors, especially among visually similar digits. Common confusions include digits with shared strokes or similar silhouettes (e.g., **4 vs 9**, **3 vs 5**, and **7 vs 9**), which appear more frequently in the decision tree matrix. The MLP significantly reduces these confusions, suggesting that it captures higher-level stroke patterns and global digit structure rather than relying on local pixel thresholds.

How to interpret the classification report. For a class k:

- **Precision:** among all samples predicted as k, the fraction that are truly k.
- **Recall:** among all true k samples, the fraction correctly predicted as k.
- **F1-score:** harmonic mean of precision and recall, measuring balanced performance.

How to interpret the confusion matrix. The diagonal entries represent correct predictions, while off-diagonal entries represent systematic mistakes. For MNIST, typical confusions often occur between visually similar digits (e.g., 4 vs 9, or 3 vs 5), and the confusion matrix reveals these patterns directly.

MNIST insights. The evaluation results provide several insights into how model structure affects performance on image data:

- **MLP learns global digit representations better.** The MLP consistently outperforms the decision tree because it can learn nonlinear combinations of pixels that represent meaningful features such as strokes, curves, and digit shapes. This enables it to correctly recognize digits even when there are variations in writing style, thickness, or minor distortions.

- **Decision trees struggle with high-dimensional pixel inputs.** Since each pixel is treated as an independent feature, decision trees rely on axis-aligned threshold splits. This leads to fragmented decision boundaries that cannot effectively capture global visual structures (e.g., distinguishing a **4** from a **9** requires understanding the overall loop/angle structure, not just individual pixel thresholds).

- **Error patterns reveal "visual similarity" limitations.** The decision tree confusion matrix shows stronger systematic mistakes between digits that share similar stroke patterns. These errors indicate that the tree often overfits local pixel coincidences rather than learning stable digit-level abstractions. In contrast, the MLP reduces these confusions significantly, suggesting more robust feature learning.

- **Performance consistency matters beyond accuracy.** While accuracy summarizes overall correctness, the classification report shows that MLP has more balanced precision/recall across nearly all classes. This means the MLP is not only more accurate but also more reliable across different digit categories, reducing class-specific weaknesses.

- **Implication for model choice.** For image recognition tasks, models that can capture nonlinear patterns and interactions (such as neural networks) are generally more suitable than tree-based models unless advanced feature engineering or ensemble methods are used.

MNIST conclusion. The MLP outperforms the decision tree because it can learn nonlinear

combinations of pixels that represent digit strokes and shapes, while decision trees rely on axis-aligned pixel thresholds and struggle to capture global visual structure.

4 Fashion-MNIST Dataset

4.1 Preparing the Dataset

Data representation. The Fashion-MNIST dataset consists of 28×28 grayscale images of clothing items, belonging to 10 categories such as T-shirt/top, trouser, pullover, dress, and sneaker. Similar to MNIST, each image is flattened into a 784-dimensional feature vector, transforming the problem into a multi-class classification task with 10 labels.

Stratified train/validation split. The original training set is shuffled and split into training (80%) and validation (20%) subsets using stratified sampling. This ensures that each clothing category maintains approximately the same proportion across all subsets, which is essential for fair hyperparameter tuning and evaluation.

Split results from the notebook.

Subset	Shape (features)	Notes
Training	$(48000, 784)$	stratified 80% of original training
Validation	$(12000, 784)$	stratified 20% of original training
Test	$(10000, 784)$	untouched test set

Class distribution check. Bar charts in the notebook confirm that all Fashion-MNIST classes remain balanced across the original training set, training split, validation split, and test set.

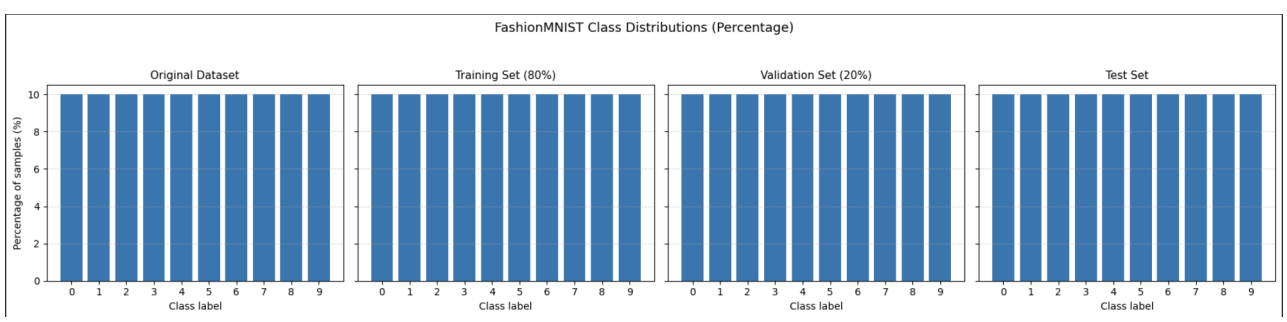

Figure 4: Fashion-MNIST class distributions across original training, train, validation, and test sets.

4.2 Building the Decision Tree Classifier

A decision tree classifier using information gain was trained:

```
DecisionTreeClassifier(criterion='entropy')
```

Challenges for Fashion-MNIST. Compared to MNIST digits, Fashion-MNIST images exhibit higher intra-class variability and greater visual similarity between different classes (e.g., shirt vs. T-shirt/top). As a result:

- Pixel-level decision boundaries are less distinctive.
- Decision trees tend to overfit local pixel patterns rather than capturing global clothing shapes.

4.3 Hyperparameter Tuning for Decision Tree

Tuning approach. A manual grid search was performed in the notebook. For each parameter combination, the model was trained on the training set and evaluated on the validation set.

Candidate hyperparameters.

$$\texttt{max_depth} \in \{10, 20, 30\}, \quad \texttt{min_samples_split} \in \{2, 20, 100\}, \quad \texttt{min_samples_leaf} \in \{1, 5\}.$$

Rationale behind the ranges.

- Larger `min_samples_split` values are tested to reduce overfitting, which is more severe for Fashion-MNIST than for MNIST.
- `min_samples_leaf` enforces more stable leaf predictions by aggregating multiple samples.

Best validation result (from notebook output).

Metric	Value	Interpretation
Best validation accuracy	**0.8109**	improved generalization over baseline DT
Best parameters	**(20, 100, 5)**	(max_depth, min_split, min_leaf)

Interpretation of best parameters. The large value of `min_samples_split=100` indicates strong regularization is necessary to avoid overfitting clothing textures and noise.

4.4 Neural Network Classifier (MLP)

Chosen MLP architecture and key hyperparameters. The implemented neural network satisfies the requirement of having at least one hidden layer, a nonlinear activation function, and a suitable optimizer. The selected configuration is:

- **Architecture:** 1 hidden layer with **128 neurons** (`hidden_layer_sizes=(128,)`)
- **Activation function: ReLU** (`activation='relu'`) to introduce nonlinearity
- **Optimization algorithm: Adam** optimizer (`solver='adam'`) for efficient gradient-based training
- **Learning rate strategy:** constant (default), with `learning_rate_init=0.001` (default)
- **Maximum training iterations:** `max_iter=200`
- **Early stopping:** enabled (`early_stopping=True`), with `n_iter_no_change=10`
- **Random seed:** `random_state=42`

Why early stopping is useful. Fashion-MNIST is more challenging than MNIST due to higher intra-class variation and multiple visually similar categories (e.g., shirts vs t-shirts, or sneakers vs ankle boots). Early stopping monitors validation performance during training and halts optimization when improvement stagnates, which reduces overfitting risk and improves training stability.

Validation result. The trained MLP achieved the following validation accuracy:

$$\textbf{Validation Accuracy (MLP)} = 0.8961$$

Although `max_iter=200` was allowed, training stopped early after **43 iterations**, indicating that the validation performance converged before reaching the iteration limit.

Additional insights. This validation score suggests that even a relatively simple MLP (one hidden layer) can learn meaningful nonlinear feature interactions from pixel-level inputs and provides a strong baseline on Fashion-MNIST. The fact that training converged after only 43 iterations also indicates that the model learns efficiently with Adam and that early stopping successfully prevents unnecessary training once generalization no longer improves. However, the accuracy remains below

typical MNIST performance because Fashion-MNIST contains more complex textures and overlapping class appearances, which require more expressive models or deeper architectures to separate perfectly.

4.5 Test Evaluation and Interpretation

Classification Report:

Decision Tree — Classification Report

	precision	recall	f1-score	support
0	0.7652	0.7920	0.7784	1000
1	0.9335	0.9270	0.9303	1000
2	0.7006	0.7300	0.7150	1000
3	0.8128	0.8380	0.8252	1000
4	0.6897	0.7090	0.6992	1000
5	0.8884	0.8840	0.8862	1000
6	0.5876	0.5100	0.5460	1000
7	0.8776	0.8820	0.8798	1000
8	0.8935	0.9140	0.9036	1000
9	0.9092	0.8910	0.9000	1000
accuracy			0.8077	10000
macro avg	0.8058	0.8077	0.8064	10000
weighted avg	0.8058	0.8077	0.8064	10000

Neural Network (MLP) — Classification Report

	precision	recall	f1-score	support
0	0.8391	0.8760	0.8571	1000
1	0.9838	0.9730	0.9784	1000
2	0.8067	0.8180	0.8123	1000
3	0.9042	0.8780	0.8909	1000
4	0.7795	0.8730	0.8236	1000
5	0.9743	0.9490	0.9615	1000
6	0.7446	0.6530	0.6958	1000
7	0.9462	0.9500	0.9481	1000
8	0.9848	0.9690	0.9768	1000
9	0.9472	0.9690	0.9580	1000
accuracy			0.8908	10000
macro avg	0.8910	0.8908	0.8903	10000
weighted avg	0.8910	0.8908	0.8903	10000

Figure 5: Fashion-MNIST Classification Reports

Observations. The classification report indicates a clear performance gap between the two models. The decision tree reaches an overall test accuracy of approximately **0.8077**, while the MLP improves to about **0.8908**. Across most classes, the MLP achieves higher precision and recall, showing more stable generalization. However, some categories remain difficult even for the MLP, especially classes with strong visual overlap such as **T-shirt/top**, **pullover**, **coat**, and particularly **shirt**, where recall is noticeably lower compared to other classes. This suggests that the main errors come from confusing visually similar upper-body clothing items.

Confusion Matrices:

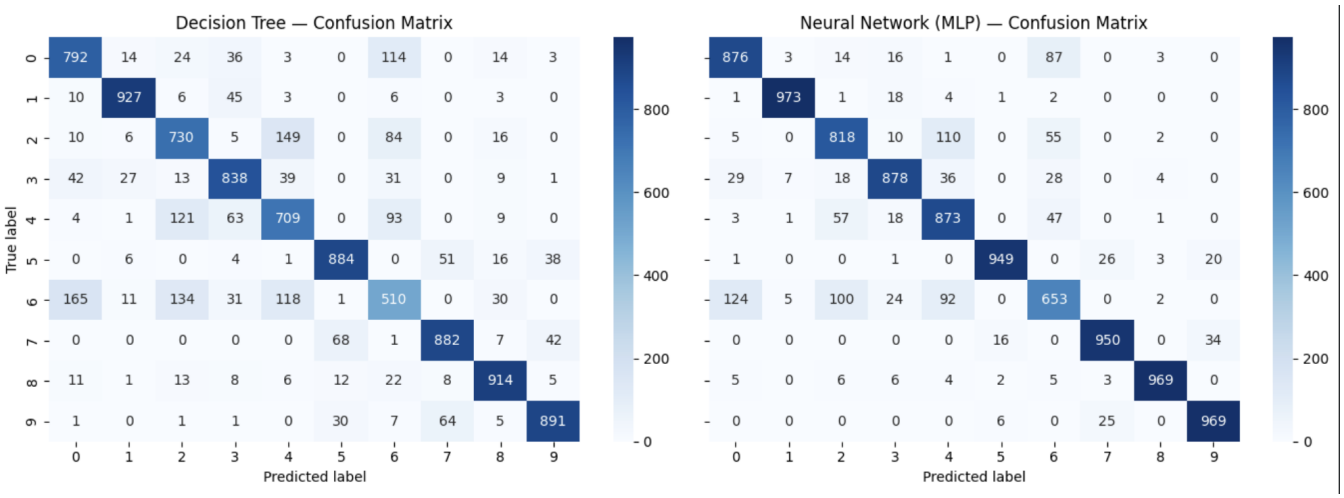

Figure 6: Fashion-MNIST Confusion Matrices

Observations. Both confusion matrices remain diagonally dominant, meaning most predictions are correct, but the decision tree contains substantially more off-diagonal mistakes. Frequent confusions occur between visually similar classes: **T-shirt/top** ↔ **shirt**, **pullover** ↔ **coat**, and **sneaker** ↔ **ankle boot**. The MLP reduces these errors significantly, producing a stronger diagonal and fewer systematic confusions, indicating that it learns more discriminative nonlinear patterns from the pixel representation. Nevertheless, the remaining mistakes suggest that certain clothing categories share highly similar silhouettes and textures, making them difficult without richer feature extraction.

Comparison between Decision Tree and MLP. Overall, the MLP outperforms the tuned decision tree on Fashion-MNIST by a large margin (about **+8%** absolute accuracy on the test set). This improvement is consistent across most classes, where the MLP achieves higher recall (fewer missed true samples) and higher precision (fewer false positives). In contrast, the decision tree struggles because its splits rely on axis-aligned pixel thresholds, which are insufficient for capturing global clothing structure and subtle texture differences. As Fashion-MNIST contains higher intra-class variation and strong inter-class similarity, tree-based boundaries become fragmented and less generalizable, even with regularization.

Fashion-MNIST insights. The evaluation provides several insights about model behavior on more complex image datasets:

- **Visual similarity dominates the error patterns.** Unlike MNIST digits, clothing categories often share similar shapes and textures. The strongest confusions occur between upper-body clothing classes (T-shirt/top, shirt, coat, pullover), showing that the main difficulty is

distinguishing subtle category boundaries.

- **MLP captures better global representations, but still has limitations.** The MLP improves performance by learning nonlinear pixel interactions that represent clothing contours and coarse texture. However, because inputs are still flattened pixels, spatial locality is not explicitly preserved, which limits its ability to separate very similar classes perfectly.
- **Decision trees overfit local pixel patterns.** The decision tree makes more systematic off-diagonal errors because it focuses on local threshold conditions that do not generalize well across writing-style variations, shading, and fabric texture. This highlights why decision trees are less suitable for high-dimensional image data without handcrafted features.
- **Implication for future improvements.** To further reduce confusions between similar clothing classes, models that exploit spatial structure (e.g., CNNs) or feature extraction methods (e.g., PCA, HOG descriptors, or convolutional embeddings) would likely provide additional gains beyond the MLP baseline.

5 Food-MNIST Dataset

5.1 Preparing the Dataset

Data representation. Food-MNIST contains RGB images of food items from 10 categories. Each image is resized and flattened into a 3072-dimensional vector ($32 \times 32 \times 3$), resulting in a much higher-dimensional and noisier feature space compared to MNIST and Fashion-MNIST.

Stratified split. The training data is split into training (80%) and validation (20%) subsets using stratified sampling, while the test set remains untouched.

Split results from the notebook.

Subset	Shape (features)	Notes
Training	$(3000, 3072)$	stratified 80%
Validation	$(750, 3072)$	stratified 20%
Test	$(1250, 3072)$	final evaluation only

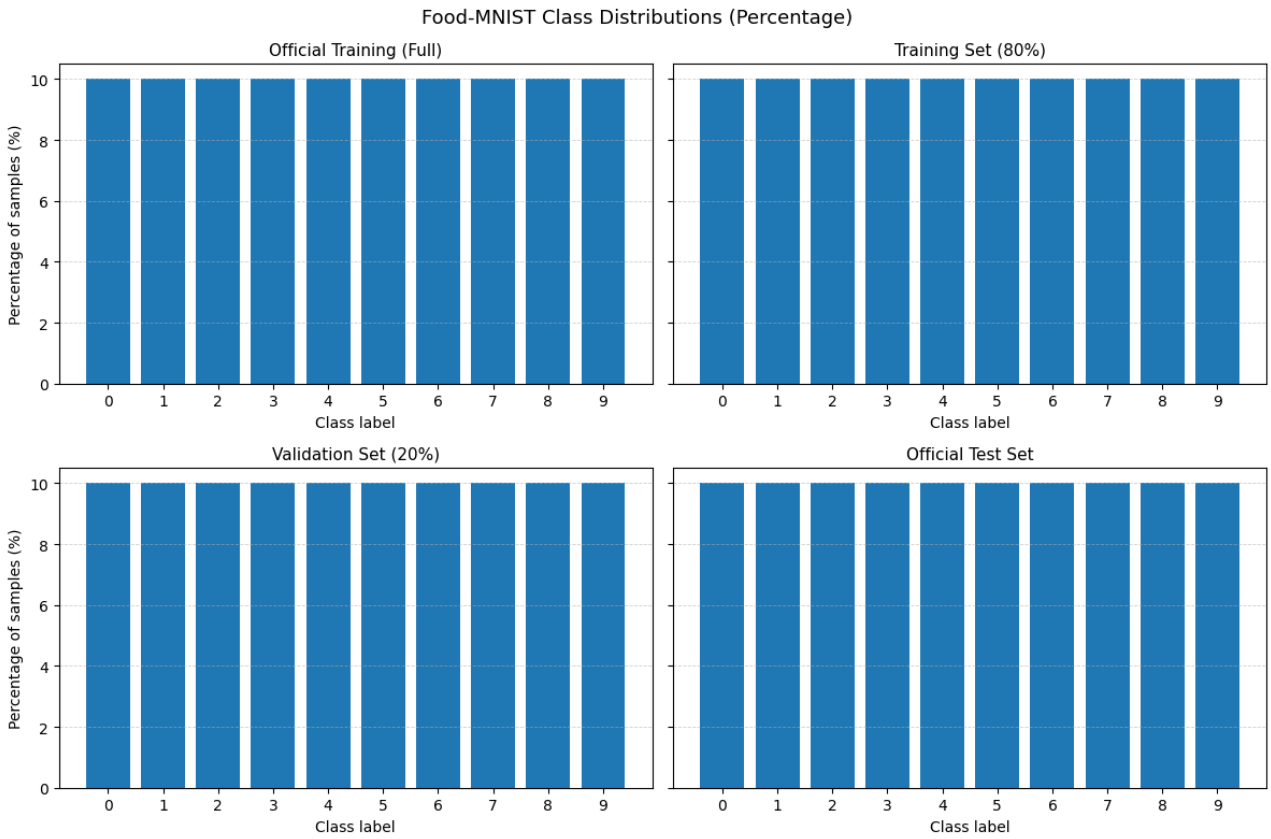

Figure 7: Food-MNIST class distributions across data splits.

5.2 Building the Decision Tree Classifier

A decision tree using entropy was trained on Food-MNIST.

Why Food-MNIST is difficult for decision trees.

- High-dimensional RGB pixels introduce strong feature correlation.
- Food images contain noise, color variation, and background clutter.
- Axis-aligned splits cannot effectively capture texture and color interactions.

5.3 Hyperparameter Tuning for Decision Tree

Grid search configuration.

$$\texttt{max_depth} \in \{10, 20, 30\}, \quad \texttt{min_samples_split} \in \{2, 20, 100\}, \quad \texttt{min_samples_leaf} \in \{1, 5\}.$$

Best validation result (from notebook output).

Metric	Value	Interpretation
Best validation accuracy	**0.1907**	limited generalization capability
Best parameters	**(10, 100, 1)**	(max_depth, min_split, min_leaf)

Interpretation. A shallow tree with strong regularization performs best, indicating that deeper trees severely overfit noise in food images.

5.4 Neural Network Classifier (MLP)

MLP configuration. We trained an `sklearn.neural_network.MLPClassifier` on the prepared training set and evaluated it on the validation set. The chosen architecture and key hyperparameters are:

- **Architecture:** 1 hidden layer with 128 neurons (`hidden_layer_sizes=(128,)`)
- **Activation function:** ReLU (`activation='relu'`)
- **Optimizer:** Adam (`solver='adam'`)
- **Learning rate strategy:** constant (default), with `learning_rate_init=0.001` (default)
- **Max iterations:** `max_iter=500`
- **Early stopping:** enabled (`early_stopping=True`), with `n_iter_no_change=10`
- **Random seed:** `random_state=42`

Validation result. The MLP achieves a low validation accuracy on this dataset:

$$\textbf{Validation Accuracy (MLP)} = 0.2800$$

Training terminated early after **27 iterations** due to early stopping, indicating that the validation score stopped improving for several consecutive iterations.

Why the MLP struggles on this dataset. This dataset uses RGB images with richer textures, backgrounds, and color variations. When images are flattened into a 1D vector, the spatial structure of the image is lost (e.g., neighboring pixels in the original image are no longer treated

as neighbors in the input representation). As a result, a fully-connected MLP must learn spatial relationships implicitly, which is difficult and inefficient for natural images. This often leads to poor generalization and unstable decision boundaries, especially when there is strong intra-class variation. In contrast, convolutional neural networks (CNNs) preserve spatial locality and learn hierarchical visual features (edges → textures → object parts), making them significantly more suitable for RGB image classification tasks.

5.5 Test Evaluation and Interpretation

Food-MNIST Classification Reports

Decision Tree — Classification Report

	precision	recall	f1-score	support
0	0.1438	0.1680	0.1550	125
1	0.2165	0.4400	0.2902	125
2	0.2400	0.1920	0.2133	125
3	0.1748	0.1440	0.1579	125
4	0.2022	0.2880	0.2376	125
5	0.1200	0.0240	0.0400	125
6	0.3168	0.4080	0.3566	125
7	0.2674	0.4000	0.3205	125
8	0.0606	0.0320	0.0419	125
9	0.0667	0.0160	0.0258	125
accuracy			0.2112	1250
macro avg	0.1809	0.2112	0.1839	1250
weighted avg	0.1809	0.2112	0.1839	1250

Neural Network (MLP) — Classification Report

	precision	recall	f1-score	support
0	0.2361	0.1360	0.1726	125
1	0.2022	0.1440	0.1682	125
2	0.3750	0.2400	0.2927	125
3	0.2757	0.4720	0.3481	125
4	0.2308	0.1200	0.1579	125
5	0.2955	0.2080	0.2441	125
6	0.3715	0.7520	0.4974	125
7	0.4785	0.6240	0.5417	125
8	0.1351	0.0400	0.0617	125
9	0.1746	0.2640	0.2102	125
accuracy			0.3000	1250
macro avg	0.2775	0.3000	0.2695	1250
weighted avg	0.2775	0.3000	0.2695	1250

Figure 8: Food-MNIST Classification Reports

Observations. The classification report shows that both models struggle on Food-MNIST due to the dataset complexity. The decision tree achieves a very low overall test accuracy of approximately **0.2112**, while the MLP improves to about **0.3000**. Although the MLP performs better across most classes (higher precision and recall), the performance is still far below what was observed on MNIST and Fashion-MNIST. This indicates that Food-MNIST contains more challenging visual variations such as diverse colors, textures, and background patterns that are difficult to capture using flattened pixel features.

Food-MNIST Confusion Matrices:

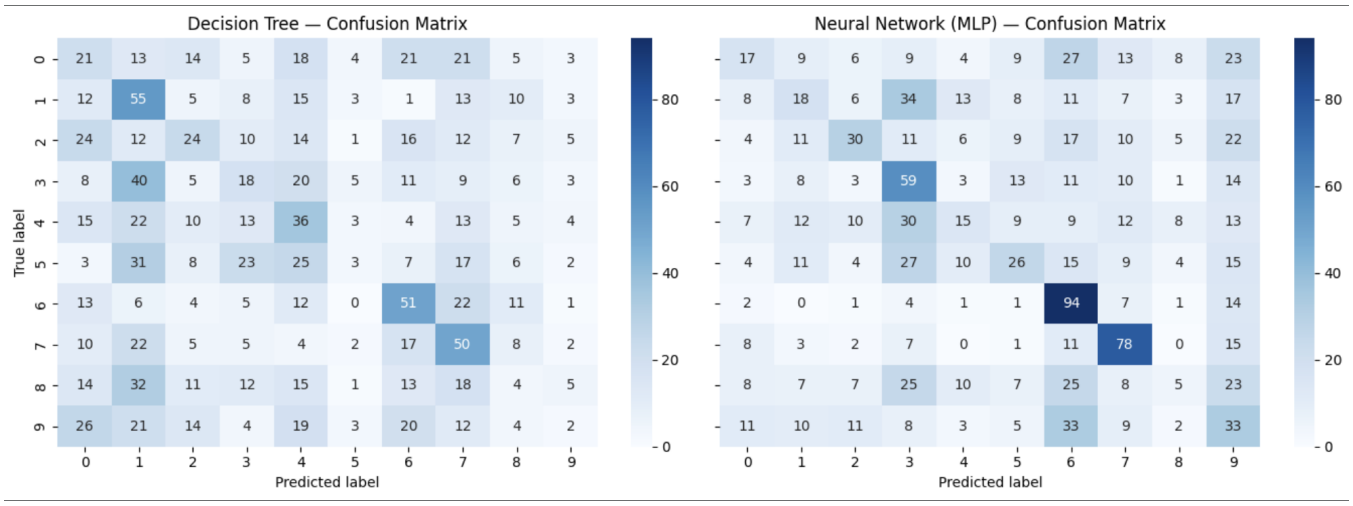

Figure 9: Food-MNIST Confusion Matrices

Observations. The confusion matrices reveal weak diagonal dominance compared to the previous datasets. Both models produce a large number of off-diagonal predictions, meaning they frequently confuse different food categories. The decision tree matrix appears highly dispersed, suggesting that the tree fails to learn stable decision boundaries in the high-dimensional RGB feature space. The MLP shows slightly stronger diagonal patterns, but still exhibits many systematic confusions, consistent with the limited accuracy reported above.

Comparison between Decision Tree and MLP. The MLP outperforms the tuned decision tree on Food-MNIST, improving test accuracy by nearly **+9%** (from **0.2112** to **0.3000**). This improvement suggests that nonlinear modeling helps in learning more informative feature interactions than axis-aligned decision rules. However, both models remain weak because they operate on flattened RGB pixel vectors, where spatial locality, texture continuity, and color composition are not explicitly preserved.

Food-MNIST insights. Food-MNIST is the most challenging dataset in this project for the following reasons:

- **High-dimensional input and stronger noise.** Each image has size $32 \times 32 \times 3 = 3072$ features, which is much higher than MNIST/Fashion-MNIST (784 features). This increases feature correlation and makes learning harder for both trees and MLPs.

- **Color and texture patterns dominate food recognition.** Unlike digits or clothing silhouettes, food images rely heavily on fine-grained texture cues (e.g., grainy patterns, smooth

surfaces) and color mixtures, which are difficult to represent using simple global pixel thresholds or fully-connected layers on flattened inputs.

- **Flattening destroys spatial structure.** When RGB images are flattened, local neighborhood information is lost. As a result, the MLP must learn spatial relationships implicitly, which is inefficient and results in limited generalization.

5.6 Bonus Task: Convolution Neural Network (CNN)

To address the limitation of flattened representations, we implemented a Convolutional Neural Network (CNN) as a bonus experiment. CNNs are designed for image data because they preserve spatial locality and learn hierarchical features such as edges, textures, and color compositions.

CNN architecture (lightweight design). To better exploit spatial structure in RGB images, we implemented a compact CNN and trained it on the reshaped Food-MNIST images. Specifically, the flattened feature vectors were reshaped back into $32 \times 32 \times 3$ tensors before training:

$$X \in \mathbb{R}^{N \times 32 \times 32 \times 3}.$$

The dataset split for the CNN experiment is: **3000 training images**, **750 validation images**, and **1250 test images**.

A lightweight CNN architecture was employed to balance performance and computational cost:

- **Input:** $32 \times 32 \times 3$ RGB images
- **Feature extractor:** three convolutional blocks, each consisting of **Convolution + ReLU + MaxPooling**
- **Classifier head:** fully-connected layer(s) followed by a **Softmax** output layer
- **Training strategy:** early stopping based on validation accuracy to prevent overfitting and unnecessary training

Result. The CNN achieved the following performance on the Food-MNIST test set:

$$\textbf{Test Accuracy (CNN)} = 45.44\%.$$

Interpretation. This result represents a substantial improvement over both the decision tree and the MLP baselines. The gain confirms that convolutional architectures are more suitable for Food-MNIST because they preserve spatial locality, learn texture-sensitive filters, and capture hierarchical visual patterns (edges → textures → food parts) directly from RGB images.

6 Model Comparison and Performance Analysis

Based on the experimental results across three datasets (MNIST, Fashion-MNIST, and Food-MNIST), clear differences can be observed in the performance of Decision Trees, Multilayer Perceptrons (MLP), and Convolutional Neural Networks (CNN).

Decision Trees achieve the lowest performance on all image-based datasets. Although the use of Information Gain enables interpretable decision boundaries, Decision Trees struggle with high-dimensional pixel inputs. Flattened image vectors lead to fragmented and axis-aligned splits, which are insufficient for capturing complex visual patterns such as edges, textures, and shapes. As a result, Decision Trees perform reasonably on MNIST but degrade significantly on Fashion-MNIST and Food-MNIST.

Multilayer Perceptrons (MLP) consistently outperform Decision Trees by learning nonlinear feature representations. On MNIST, MLP achieves high validation accuracy, demonstrating its effectiveness when digit shapes are relatively simple and well-aligned. However, when applied to Fashion-MNIST and especially Food-MNIST, MLP performance is limited. Flattening RGB images removes spatial locality, preventing the network from modeling texture and color correlations effectively. Increasing the number of training iterations does not fully resolve this limitation, leading to slow convergence and suboptimal accuracy.

Convolutional Neural Networks (CNN) provide the best performance on the most complex dataset, Food-MNIST. By preserving spatial structure through convolutional layers and local receptive fields, CNNs are able to capture hierarchical visual features such as edges, patterns, and object parts. Although the CNN used in this project is relatively shallow, it still significantly improves over MLP and Decision Trees on Food-MNIST. Early stopping further stabilizes training and prevents overfitting by selecting the best-performing model on the validation set.

7 Conclusions

Overall, the results demonstrate that *model suitability strongly depends on data structure*. Decision Trees offer interpretability but are not suitable for high-dimensional image data. MLPs provide improved nonlinear modeling but suffer from loss of spatial information. CNNs are the most appropriate choice for complex visual datasets due to their ability to exploit spatial locality and hierarchical feature learning.

References

[1] CVD Foundation. mnist. https://github.com/cvdfoundation/mnist.

[2] Faculty of Information Technology, University of Science, VNU-HCM. Cs420 artificial intelligence – project 2: Machine learning. Project specification document.

[3] srohit0. food_mnist. https://github.com/srohit0/food_mnist.

[4] Zalando Research. fashion-mnist. https://github.com/zalandoresearch/fashion-mnist.